条件分支与循环

- 来源: https://class.imooc.com/lesson/5285#mid=105037
- 阶段: 阶段四: 【框架应用】高效AI开发
- 周: 第9周 LangGraph：工作流编排
- 模块: 文章写作Agent
- 课节: 图文 1-2 条件分支与循环

1-2 条件分支与循环

条件分支与循环

> [!TIP] 阅读指南 上一节我们构建了一个简单的文章写作工作流：大纲 → 草稿 → 润色，三步顺序执行。但在实际应用中，我们常需要工作流“会思考、会判断”： 生成的草稿质量不够？循环重试 → 回到生成节点。 用户问题类型不同？条件分支 → 选择不同路径。 检测到敏感内容？转人工审核 → 跳转特定节点。 本节将学习两个核心机制： 循环 (Loop)：指向前面的节点，重复执行。 分支 (Branch)：指向不同后续节点，分流处理。

从固定流程到动态决策

第3节的局限

回顾第3节的工作流：

```
START → 大纲 → 草稿 → 润色 → END

1
```

1

这是一条固定的单行道：

固定的单行道

- [√] 流程清晰，执行可预测。

- [×] 无法根据中间结果调整路径。
- [×] 草稿质量差也只能继续润色。
- [×] 无法处理异常情况。

两种动态机制

实际项目中，我们需要两种能力：

循环能力：向后指，重复执行

循环能力

循环特征：

循环特征

- 质量检查 → 草稿：指向前面的节点。

```
质量检查 → 草稿
```

- 形成有向有环图。

有向有环图

- 节点可能执行多次。

执行多次

- 必须有退出条件（避免死循环）。

退出条件

分支能力：向前指，分流处理

分支能力

分支特征：

分支特征

- 分类 → [技术/闲聊/人工]：指向不同后续节点。

```
分类 → [技术/闲聊/人工]
```

- 形成有向无环图 (DAG)。

有向无环图 (DAG)

- 每个节点最多执行一次。

最多执行一次

- 多条路径互不交叉，各自通向终点。

互不交叉

条件边 (Conditional Edges)

核心概念

条件边允许根据 State 动态选择下一个节点：

条件边

```
# 固定边：永远走同一条路
builder.add_edge("A", "B")  # A → B

# 条件边：根据 State 选择路径
builder.add_conditional_edges(
    "A",                    # 起始节点
    routing_function,       # 路由函数(判断逻辑)
    {                       # 路由映射表
        "路径1": "B",
        "路径2": "C",
        "路径3": END
    }
)
```

路由函数示例：

```
def routing_function(state: MyState) -> str:
    """根据 State 返回路径名称"""
    if state["score"] > 80:
        return "路径1"  # 走向节点 B
    elif state["score"] > 60:
        return "路径2"  # 走向节点 C
    else:
        return "路径3"  # 结束流程
```

实战案例 1：循环重试机制

需求分析

场景：文章写作 Agent 的质量自动检查。

场景

核心需求：实现循环重试机制。

核心需求

- 草稿生成后，自动检查质量。
- 质量合格（字数 800-1500） → 进入润色。
- 质量不合格 → 循环回到草稿节点重新生成。

循环回到草稿节点

- 最多重试 3 次，超限则强制进入润色。

> 完整源码路径: samples/chapter9/article_writer/article_writer_with_loop.py

工作流结构可视化：

工作流结构可视化

关键代码实现

在 LangGraph v0.2+ 中，我们使用 START 和 END 常量来定义图的边界。以下是核心编排逻辑：

```
from langgraph.graph import StateGraph, START, END

# 1. 路由函数：根据字数和重试次数决定路径
def check_quality_route(state: ArticleState) -> str:
    draft = state["draft"]
    retry_count = state.get("retry_count", 0)
    word_count = len(draft)

    if 800 <= word_count <= 1500:
        return "polish"  # 合格 -> 润色
    elif retry_count >= 3:
        return "polish"  # 达到重试上限 -> 强制进入润色
    else:
        return "retry"   # 不合格 -> 重试

# 2. 构建图
builder = StateGraph(ArticleState)

builder.add_node("大纲", generate_outline)
builder.add_node("草稿", write_draft)
builder.add_node("质量检查", quality_check)
builder.add_node("润色", polish_article)

# 定义流程
builder.add_edge(START, "大纲")
builder.add_edge("大纲", "草稿")
builder.add_edge("草稿", "质量检查")

# 添加条件边实现循环
builder.add_conditional_edges(
    "质量检查",
    check_quality_route,
    {
        "retry": "草稿",   # 路由返回 "retry" 时回到 "草稿" 节点
        "polish": "润色"  # 路由返回 "polish" 时进入 "润色" 节点
    }
)

builder.add_edge("润色", END)
```

实战案例 2：条件分支路由

场景：智能客服系统的分类分发。

核心需求：实现多路径分支。

多路径分支

- 技术问题 → 走向 查文档 节点。

```
查文档
```

- 闲聊问候 → 走向 闲聊回复 节点。

```
闲聊回复
```

- 无法分类 → 走向 转人工 节点。

> 完整源码路径: samples/chapter9/article_writer/question_router.py

分支路由通常用于将任务分发给不同的专业处理节点：

```
def question_router(state: CustomerState) -> str:
    category = state["category"]
    if "技术" in category:
        return "technical"
    elif "闲聊" in category:
        return "casual"
    else:
        return "human"

builder = StateGraph(CustomerState)

# 添加节点...
builder.add_node("分类", classify_question)
builder.add_node("查文档", search_docs)
builder.add_node("闲聊", casual_reply)
builder.add_node("转人工", transfer_human)

builder.add_edge(START, "分类")

# 添加条件分支
builder.add_conditional_edges(
    "分类",
    question_router,
    {
        "technical": "查文档",
        "casual": "闲聊",
        "human": "转人工"
    }
)

# 所有分支最终汇聚到结束
builder.add_edge("查文档", END)
builder.add_edge("闲聊", END)
builder.add_edge("转人工", END)
```

■ 学点英语

中文	English	音标	说明
条件边	Conditional Edge	/kənˈdɪʃənəl edʒ/	根据 State 动态决定下一步的边，实现分支和循环
路由函数	Routing Function	/ˈruːtɪŋ ˈfʌŋkʃn/	读取 State 返回路径名称的判断逻辑
重试机制	Retry Mechanism	/ˈriːtraɪ ˈmekənɪzəm/	条件循环 + 最大次数约束的容错模式
有向无环图 vs 有向有环图	DAG vs Cyclic Graph	/diː eɪ dʒiː ˈvɜːrsəs ˈsaɪklɪk græf/	分支形成 DAG，循环形成有环图

下节预告

到目前为止，我们构建的都是全自动工作流：一旦启动，Agent 会按照预设的路径一路执行到底。但在真实的生产环境中，这种"完全自动化"往往是危险的。

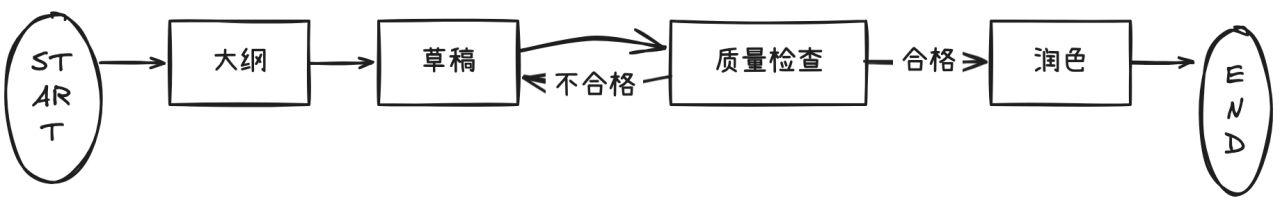
回想一下第3节的文章写作 Agent。虽然我们实现了大纲生成、草稿撰写、润色优化的完整流程，但有一个关键问题：AI 生成的每一步内容，真的可以直接用吗？

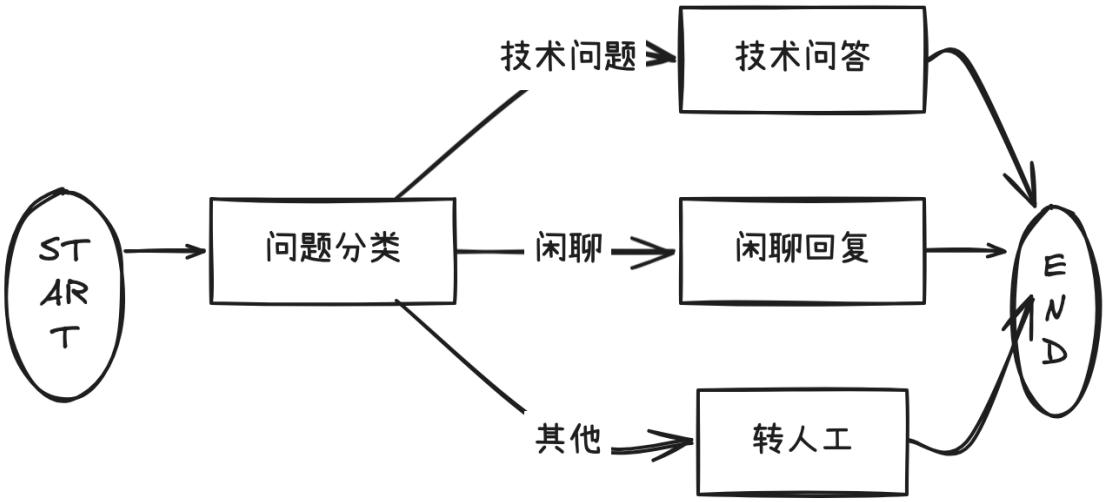
答案是否定的。真正可靠的工作流应该是：LLM 生成 → 人工审核/修改 → 继续执行。

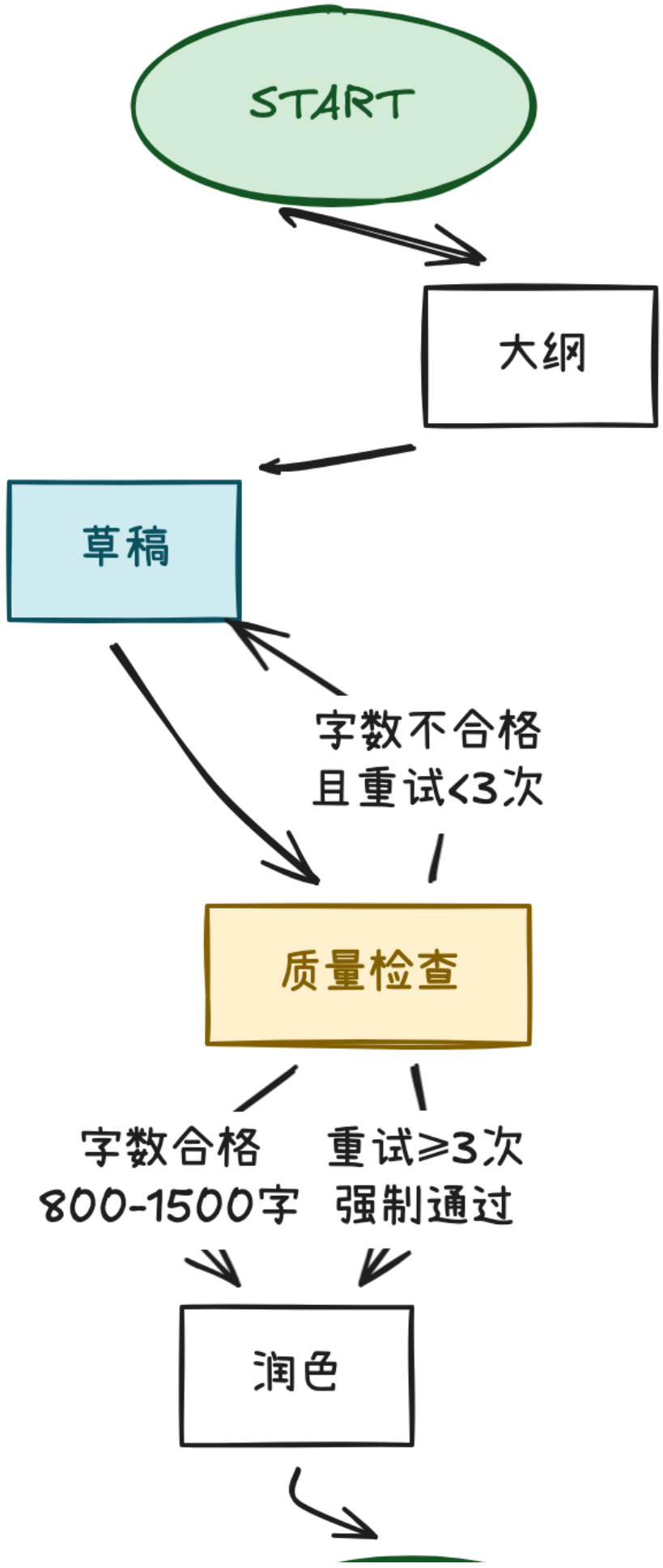
下一节，我们将学习 LangGraph 最强大的特性之一——人工介入（Human-in-the-Loop）：

- 如何在关键节点暂停工作流，等待人工决策。
- 如何在人工审核后，继续执行后续流程。
- 如何实现「AI 高效生成 + 人类精准把关」的完美协作。

图片




START
问题分类
技术问题
技术问答
闲聊
闲聊回复
其他
转人工
END


START
大纲
草稿
字数不合格
且重试<3次
质量检查
字数合格
800-1500字
重试≥3次
强制通过
润色


END